

운영 중인 서비스의 UI 구조와 유지보수를 고려하는 웹퍼블리셔 최은혜입니다.

웹퍼블리셔 6년 · 교육, 커머스, 게임, 간편결제, 웹 에디터 서비스의 UI 구축과 운영

- HTML5
- CSS3 / SCSS
- JavaScript
- jQuery
- React
- Vite / Gulp
- Git / SVN
- 웹접근성

이 포트폴리오에서 다루는 것

- Case 1

레거시 UI 코드 구조 개선

기능별 분리로 수정 범위를 파악하기 쉽게
- Case 2

아토믹 디자인 기반 컴포넌트

신규 사이트의 반복 UI를 단계별로 구성
- Case 3

웹접근성 개선 · 비디오 모듈

키보드 접근, 대체 텍스트, 포커스 처리

“재현 예제”는 회사 코드를 쓰지 않고 업무에서 다룬 패턴을 새로 작성한 것입니다. 실제 화면과 코드는 공개 가능한 범위만 다룹니다.

Case 1. 레거시 UI 코드의 기능별 구조 개선

회사 / 기간 (주)모두의교육그룹 · 2024.07 ~ 2025.10 (주)바른컴퍼니 · 2023.08 ~ 2024.02	담당 / 기여도 웹퍼블리셔 · 모두의교육그룹 1명 100%, 바른컴퍼니 2명 80%	개발환경 HTML, CSS, SCSS, PHP, JavaScript, jQuery
---	---	---

배경과 문제

프로모션 페이지와 패밀리 사이트를 운영하면서 기존 레거시 UI 코드를 계속 수정해야 했습니다. 수정 범위를 파악하기 어려우면 작은 변경도 다른 화면에 영향을 줄 수 있어, 코드를 기능별로 나누고 구조화하는 작업이 필요했습니다.

제가 한 일

- 기존 레거시 코드를 분석하고 기능별로 분리하여 수정과 유지보수가 쉬운 구조로 개선했습니다.
- 사용자 편의성을 고려한 UI 개선과 운영 중 발생하는 UI 이슈 대응을 함께 담당했습니다.

구조 개선 방식

재구성 예시

아래는 실제 파일이 아니라, 기능 단위 분리 방식을 이해하기 쉽게 다시 그린 예시입니다.

Before

```
css/  
  style.css      /* 모든 페이지 스타일이  
한 파일 */  
js/  
  common.js      /* 페이지별 동작이  
뒤섞임 */
```

After

```
scss/  
  base/          /* reset, 변수 */  
  components/    /* button, form,  
card ... */  
  pages/         /* 페이지별 스타일 */  
js/  
  modules/       /* 기능별 모듈 (탭,  
팝업 ...) */
```

운영 사이트

- modulearning.kr
- modube.kr
- krcls.kr
- barunsoncard.com, premierpaper.co.kr, deardeer.kr

개선 전후의 실제 화면과 코드는 공개 가능 여부를 확인한 뒤 추가할 수 있습니다.

Case 2. 아토믹 디자인 기반 컴포넌트 구조화

회사 / 기간

(주)모두의교육그룹 ·
2024.07 ~ 2025.10

담당 / 기여도

웹퍼블리셔 · 1명 100%

개발환경

HTML, CSS, SCSS,
JavaScript, Vite

배경과 문제

신규 사이트를 제작하면서 폼 그룹, 리스트, 카드, 댓글처럼 여러 화면에서 반복되는 UI를 화면마다 따로 만들면, 수정할 때 모든 곳을 찾아 고쳐야 합니다. 그래서 UI를 **아토믹 디자인** 방식으로 나누어 구성했습니다.

제가 한 일

- 신규 사이트 제작에 아토믹 디자인 방식을 적용해 UI를 재사용 가능한 컴포넌트 구조로 구성했습니다.
- 폼 그룹, 리스트, 카드, 댓글 등 반복 UI를 컴포넌트화하고 HTML·SCSS·JavaScript를 기능 단위로 모듈화했습니다.
- SCSS 폴더 구조를 역할별로 세분화하고, 가능한 한 컴포넌트 단위로 스타일 코드를 재사용하도록 구성했습니다.
- 스타일과 구조를 바꿀 때 한 곳만 고치면 되도록 코드를 구조적으로 작성해, 유지보수가 원활하도록 했습니다.
- Vite와 SCSS 기반 개발 환경에서 UI와 스타일 코드를 관리했습니다.

해당 사이트는 이후 디자인이 전면 개편되어 현재 공개된 화면에서는 당시 구조를 확인할 수 없습니다. 아래는 같은 방식을 새로 작성한 예제입니다.

아토믹 디자인 구조

재현 예제

가장 작은 단위(Atoms)를 조합해 Molecules, Organisms로 올라가는 구조입니다. 아래 화면은 같은 원자를 조합해 만든 결과입니다.

Atoms

더 쪼갤 수 없는 요소

이름

입력창

버튼

김



Molecules

원자의 조합

이름

라벨 + 입력창 + 안내 문구

이 이OO

아바타 + 이름 + 내용



Organisms

분자의 조합

댓글 영역

김 김OO

댓글 목록

이 이OO

같은 분자를 반복합니다

댓글

등록

SCSS 폴더 구조 재구성 예시

스타일을 역할별 폴더로 나누고, 반복되는 UI는 `components/` 안에서 아토믹 단계별로 분리하는 구조입니다.

```
scss/  
  abstracts/      _variables.scss _mixins.scss _functions.scss  
  vendors/        외부 라이브러리 스타일  
  base/           _reset.scss   _typography.scss  
  layout/         _header.scss  _footer.scss  _grid.scss  
  components/  
    atoms/        _button.scss  _input.scss  _label.scss  _avatar.scss  
    molecules/    _form-field.scss _comment.scss  
    organisms/    _comment-section.scss _card-list.scss  
  pages/          _home.scss   (페이지 전용 스타일만)  
  style.scss      부분 파일을 모아 불러오는 메인 파일
```

- `abstracts/` : 직접 컴파일되지 않고 다른 스타일에서 가져다 쓰는 변수, 믹스인, 함수
- `base/` , `layout/` : 리셋과 기본 태그 스타일, 헤더·푸터 같은 큰 틀
- `components/` : 독립적으로 재사용되는 UI 조각. 아토믹 단계별로 분리
- `pages/` : 특정 페이지에서만 쓰는 스타일

```
// style.scss : 부분 파일(_로 시작)을 한곳에서 불러와 하나의 CSS로 컴파일  
@use 'abstracts/variables' as *;  
@use 'base/reset';  
@use 'layout/header';  
@use 'components/atoms/button';  
@use 'components/molecules/form-field';  
@use 'components/organisms/comment-section';  
@use 'pages/home';
```

각 단계는 한 방향으로만 조합합니다. 수정할 때 영향 범위가 아래 단계에서 위 단계로만 번지므로, 입력창 하나를 고치면 그 입력창을 쓰는 모든 화면이 함께 바뀌고 다른 요소는 건드리지 않게 됩니다.

그 외 공통 UI 경험

- NAVER Smart Editor(4명 75%): React 기반 프로젝트에서 재사용 가능한 UI 컴포넌트를 작업하고 관리했습니다.
- NHN 서비스(5명 50%): 공통 UI 컴포넌트를 개발·관리하고 유지보수를 고려한 UI 작업 프로세스를 개선했습니다.

Case 3. 웹접근성 개선과 학습용 비디오 모듈 UI

회사 / 기간

(주)모두의교육그룹 ·
2024.07 ~ 2025.10

담당 / 기여도

웹퍼블리셔 · 1명 100%

개발환경

HTML, CSS, SCSS,
JavaScript, jQuery,
Video.js

배경과 문제

학습 서비스에서는 마우스를 쓰기 어려운 사용자도 같은 기능을 이용할 수 있어야 합니다. 화면이 보이는 대로 동작하는 것만으로는 부족해서, 키보드 접근과 대체 텍스트, 포커스 처리를 기준으로 UI 구조와 인터랙션을 점검했습니다.

제가 한 일

- 웹접근성을 고려해 UI 구조와 인터랙션을 개선했습니다. (키보드 접근, 대체 텍스트, 포커스 처리)
- Video.js를 서비스 특성에 맞게 커스터마이징하여 학습용 비디오 모듈 UI를 작업했습니다.
- 강의마다 퀴즈 유형이 하나씩만 연결되는 구조에 맞춰, JSON 데이터의 퀴즈 값에 따라 해당 퀴즈의 메뉴 인덱스와 퀴즈 HTML을 불러오도록 JavaScript를 작업했습니다.

퀴즈 유형별 메뉴와 HTML 불러오기 재현 예제

각 강의에는 퀴즈가 한 종류만 올 수 있습니다. 강의 데이터(JSON)에 `quiz` 값을 두고, 이 값으로 비디오 모듈의 **메뉴 위치(인덱스)**와 **퀴즈 HTML 파일**을 정하도록 했습니다. 예를 들어 `quiz: "ox"` 이면 OX 퀴즈가 들어가는 메뉴 인덱스와 OX 퀴즈 HTML을 불러옵니다. 퀴즈 유형이 늘어도 데이터와 매핑만 추가하면 되고, 퀴즈가 없는 강의는 값이 `null` 이라 퀴즈 메뉴가 나오지 않습니다.

OX 퀴즈 quiz: "ox"

객관식 퀴즈 quiz: "choice"

퀴즈 없음 quiz: null

강의 안내 화면입니다.

예제 화면은 실제 HTML 파일 대신 `template` 요소로 동작을 대신 보여 줍니다. 아래 코드는 이 구조를 설명하기 위해 새로 작성한 예시이며, 실제 서비스 코드가 아닙니다.

```
// 강의 데이터 (JSON)
{
  "lectures": [
    { "id": "lec01", "quiz": "ox" },
    { "id": "lec02", "quiz": "choice" },
    { "id": "lec03", "quiz": null }
  ]
}
```

```
// 퀴즈 유형별 메뉴 인덱스와 HTML 파일 매핑
const QUIZ_MAP = {
  ox: { menuIndex: 2, html: 'quiz/ox.html' },
  choice: { menuIndex: 2, html: 'quiz/choice.html' }
};

async function applyQuiz(lecture) {
  const conf = QUIZ_MAP[lecture.quiz]; // quiz 값으로 설정 선택
  if (!conf) { removeQuizMenu(); return; } // 퀴즈가 없는 강의

  insertQuizMenu(conf.menuIndex); // 해당 메뉴 인덱스에 퀴즈 메뉴 추가
  const res = await fetch(conf.html); // 퀴즈 HTML 파일 불러오기
  quizPanel.innerHTML = await res.text();
}
```

재현 예제 2: 키보드로 이용하는 팝업 직접 작성

팝업을 열면 포커스가 팝업 안으로 이동하고, Esc로 닫으면 팝업을 연 버튼으로 돌아옵니다.

팝업 열기

```
opener.addEventListener('click', () => dialog.showModal()); // 포커스가 팝업 안으로 이동
dialog.addEventListener('close', () => opener.focus()); // 닫으면 열었던 버튼으로 복귀
```

재현 예제 3: 재생 컨트롤의 이름과 상태

직접 작성

아이콘만 있는 버튼도 스크린 리더가 읽을 수 있는 이름이 있어야 하고, 재생 상태는 `aria-pressed` 로 전달합니다. 실제 영상 대신 타이머로 동작을 흉내 낸 예제입니다.



```
<button type="button" aria-pressed="false" aria-label="재생">
  <svg aria-hidden="true" focusable="false">...</svg>
</button>
```

재현 예제 4: 이미지의 대체 텍스트

직접 작성

의미가 있는 이미지

```

```

내용을 대신 설명하는 문장을 넣습니다.

장식용 이미지

```

```

정보가 없으면 빈 `alt` 로 읽지 않게 합니다.